



МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования

«МИРЭА Российский технологический университет»
РТУ МИРЭА

Контрольная работа № 5

Выполнила студентка 2 курса
Группы ЭФБО-05-24 Гузаирова О.Р.

Москва

2026 г.

Оглавление

1. Введение	4
1.1 Цель	4
1.2 Область действия (Scope)	4
1.3 Определения и аббревиатуры	5
2. Описание	6
2.1 Описание продукта	6
2.2 Функции продукта	6
2.2.1 Создание холста	6
2.2.2 Создание точки	6
2.2.3 Создание линии	7
2.2.4 Создание прямоугольника	7
2.2.5 Заливка холста	7
2.2.6 Очищение холста	7
2.2.7 Сохранение созданного рисунка	7
2.3 Характеристики пользователей	7
3. Детальные (специфические) требования	8
3.1 Методология по сбору требований	8
3.2 Требования к проекту	8
3.2.1 Бизнес-требования	8
3.2.2 Пользовательские требования	8
3.2.3 Функциональные требования	9
3.2.4 Нефункциональные требования	9
4. Диаграммы UML	10
4.1 Диаграмма вариантов использования (Use Case Diagram)	10
4.2 Sequence diagram	10
4.3 Activity diagram	11
4.4 Class diagram	12
5. Паттерны	12
5.1 Паттерн Command	12
5.2 Паттерн Factory	13
5.3 Паттерн Facade	13

6. Тестирование системы.....	13
6.1 Unit-тесты.....	13
6.1.1 Класс Canvas	13
6.1.2 Команды.....	14
6.1.3 Класс FileManager.....	14
6.2 Тестовые сценарии.....	14
6.2.1 Тестирование рисования	14
6.2.2 Тестирование сохранения и загрузки.....	14
6.2.3 Тестирование изменения размера	15
7. Dockerfile для автоматической сборки проекта.....	15

1. Введение

В наши дни многие в условиях развития цифровых технологий все больше интересуются изучением алгоритмов и цифровой графики. Простые графические приложения помогают лучше понимать работу определенных алгоритмов и базовые принципы объектно-ориентированного программирования. Также они могут быть полезны в понимании построения удобного и простого пользовательского интерфейса, что может понадобиться при реализации последующих проектов, в которых будет использоваться его реализация.

1.1 Цель

Целью данного проекта является создание консольного приложения для создания ASCII-рисунков на языке программирования C++.

1.2 Область действия (Scope)

Данный проект представляет собой разработку простого приложения Paint, написанного на языке программирования C++ для создания простых ASCII-изображений с помощью ввода команд в консоль.

В область действия приложения входят и реализуются следующие функции:

1. Создание и изображение текстового холста с заданным пользователем размером;
2. Рисование базовых примитивов (точек, линий и прямоугольников);
3. Реализация алгоритма заливки определенной области холста (fill);
4. Реализация алгоритма полной очистки холста (erase);
5. Реализация алгоритма сохранения рисунка в файл txt;
6. Реализация алгоритма загрузки рисунка в файл txt;
7. Обработка пользовательских команд;
8. Обработка ошибок ввода;
9. Вывод результата в консоль;

В область действия приложения не входят действия и не рассматриваются следующие возможности:

1. Разработка графического пользовательского интерфейса (GUI);
2. Использование сторонних библиотек для графики (Qt);
3. Поддержка работа курсора;
4. Реализация сложных графических эффектов и анимации;

1.3 Определения и аббревиатуры

Аббревиатуры:

1. ASCII (American Standard Code for Information Interchange) - американский стандартный код для обмена информацией, который сопоставляет числовые коды (от 0 до 127) печатным символам (латиница, цифры, знаки) и управляющим командам.
2. ASCII-арт - вид изобразительного искусства, в котором картинки создаются с использованием печатных символов ASCII.
3. CLI (Command Line Interface) - это интерфейс командной строки, способ взаимодействия с компьютером или программой, при котором пользователь вводит текстовые команды, а система выполняет их.

Термины:

1. Консольное приложение - программа без графического интерфейса, которая взаимодействует с пользователем исключительно через текст в командной строке.
2. Холст (Canvas) - двумерная область, представленная в виде массива символов, на которой выполняется рисование.
3. Координаты (x, y) - пара чисел, определяющая точное положение точки на холсте.
4. Точка (Point) – элемент холста, представленный одним символом.
5. Линия (Line) – последовательность точек между двумя координатами.
6. Прямоугольник (Rectangle) - параллелограмм, где противоположные стороны попарно параллельны и равны.
7. Заливка (fill, flood fill) – алгоритм заполнения замкнутой области заданным символом.
8. Команда (Command) – текстовая инструкция, вводимая пользователем для выполнения действий.
9. C++ (си-плюс-плюс) - компилируемый язык программирования высокого уровня общего назначения с поддержкой объектно-ориентированного программирования.
10. Алгоритм - четкая, пошаговая последовательность инструкций или логическая схема, описывающая процесс решения конкретной задачи или достижения цели.
11. Ошибка ввода - сбой, возникающий, когда программа получает данные в неверном формате, не может прочитать файл или устройство (I/O error).

2. Описание

В данном разделе будет приведено полное описание создаваемого продукта со всеми возможными функциями и характеристиками пользователя.

2.1 Описание продукта

Разрабатываемый продукт представляет собой консольное приложение, написанное на языке программирования C++, предназначенное для создания и редактирования ASCII-рисунков с использованием текстовых символов.

Система ориентирована на пользователей, которые только знакомятся с программированием, а также для тех, кто интересуется созданием ASCII рисунков.

Программа предоставляет возможность создавать чистый холст, размеры которого задаются пользователем; выбирать символы, которые будут использованы в дальнейшем при рисовании; рисовать простые геометрические фигуры: точки, линии и прямоугольники; выполнять заливку областей; полностью очищать холст; сохранять созданный рисунок в txt формате; загружать рисунки в консоль.

Взаимодействие с программой осуществляется с помощью интерфейса командной строки. Пользователь вводит команды, которые обрабатываются системой, затем результат выводится в консоли в виде обновленного изображения.

2.2 Функции продукта

2.2.1 Создание холста

Система создает пустую область для рисования заданного пользователем размера. Принимает входными данными ширину и высоту холста. В результате инициализируется холст, заполненный символами по умолчанию.

2.2.2 Создание точки

Система меняет символ холста на выбранный символ в указанной позиции холста. Принимает входными данными координаты точки по x и y. В результате символ по соответствующим координатам меняется на выбранный пользователем.

2.2.3 Создание линии

Система строит линию между двумя точками на холсте. Принимает входными данными координаты точек по x и y . В результате линия отображается на холсте.

2.2.4 Создание прямоугольника

Система строит прямоугольник по заданным координатам. Принимает входными данными координаты двух противоположных углов. В результате прямоугольник отображается на холсте.

2.2.5 Заливка холста

Система заполняет выбранным символом заданную площадь холста. Принимает входными данными координаты начальной точки, размер заливки по x и y и символ, которым заполняется область. В результате область заполняется выбранными символами.

2.2.6 Очищение холста

Система возвращает все измененные символы в знаки, которые изначально были на холсте. Входные данные отсутствуют. В результате весь холст становится пустым.

2.2.7 Сохранение созданного рисунка

Система сохраняет рисунок в заданной папке в файл формата `txt`. Входные данные отсутствуют. В результате весь холст с созданным рисунком сохраняется на устройстве.

2.2.8 Загрузка созданного до этого рисунка

Система загружает файл с до этого созданным рисунком в холст. Входные данные отсутствуют. В результате на новый холст загружается до этого созданный рисунок.

2.3 Характеристики пользователей

Разрабатываемое консольное приложение ориентировано на следующие категории пользователей:

1. Студенты с начальным и средним уровнем подготовки, которые изучают основы программирования;
2. Начинающие разработчики, которые обладают базовыми знаниями программирования и средним уровнем подготовки;

3. Пользователей без технического опыта с начальным уровнем подготовки, которые заинтересованы в создании простых ASCII рисунков.

3. Детальные (специфические) требования

3.1 Методология по сбору требований

Сбор требований к разрабатываемой системе осуществлялся с использованием следующих методов:

1. Анализ предметной области – изучение работы простых консольных приложений на языке программирования C++, графических редакторов и ASCII рисунков;
2. Формирование пользовательских сценариев – определение всех действий, который может выполнять пользователь во время использования программы;
3. Постановка учебных целей – определение требований на основе задач, которые надо выполнить по мере создания программы и изучения алгоритмов ее работы.

3.2 Требования к проекту

3.2.1 Бизнес-требования

Разработка консольного приложения ASCII Paint на языке C++ направлена на создание инструмента для построения и редактирования простых графических изображений в текстовом (ASCII) формате. Приложение предназначено для демонстрации принципов обработки графических данных и организации пользовательского взаимодействия в консольной среде, а также предоставляет возможность рисования базовых геометрических фигур, редактирования холста и сохранения результатов пользователя.

3.2.2 Пользовательские требования

Пользователь должен иметь возможность:

1. создавать новый холст для рисования;
2. задавать размеры холста для рисования;
3. выбирать рисование прямоугольников, линий и точек;
4. выбирать символы для рисования;
5. выбрать заливку определенной области холста или весь холст;
6. очищать холст;
7. сохранять созданный рисунок в файл формата .txt;

8. загружать рисунок из файла формата .txt.

3.2.3 Функциональные требования

Система должна:

1. При вводе команды «create window size x y» устанавливается размер холста для рисования;
2. При вводе команды «symbol _» устанавливается символ, которым будет в дальнейшем рисовать пользователь;
3. При вводе команды «point x y» устанавливать точку в позиции (x, y);
4. При вводе команды «line x1 y1 x2 y2» рисовать линию между точками с заданными координатами;
5. При вводе команды «rect x1 y1 x2 y2» рисовать прямоугольник;
6. При вводе команды «fill x y _» заливать область символом _;
7. При вводе команды «save» сохранять файл в папке с проектом в формате txt;
8. При вводе команды «load» извлечение сохраненного файла в консоль;
9. При вводе команды «erase» очищать весь холст;
10. Выводить холст после каждой команды;
11. Обрабатывать неверные команды и выводить сообщения об ошибке.

3.2.4 Нефункциональные требования

1. Программа должна работать в стандартной консоли без графического интерфейса;
2. Время отклика между командами не должно превышать 1 секунду;
3. Программа не должна завершаться ошибкой при некорректном вводе;
4. Интерфейс должен быть текстовым и понятным пользователю;
5. Код программы должен быть структурированным и читаемым.

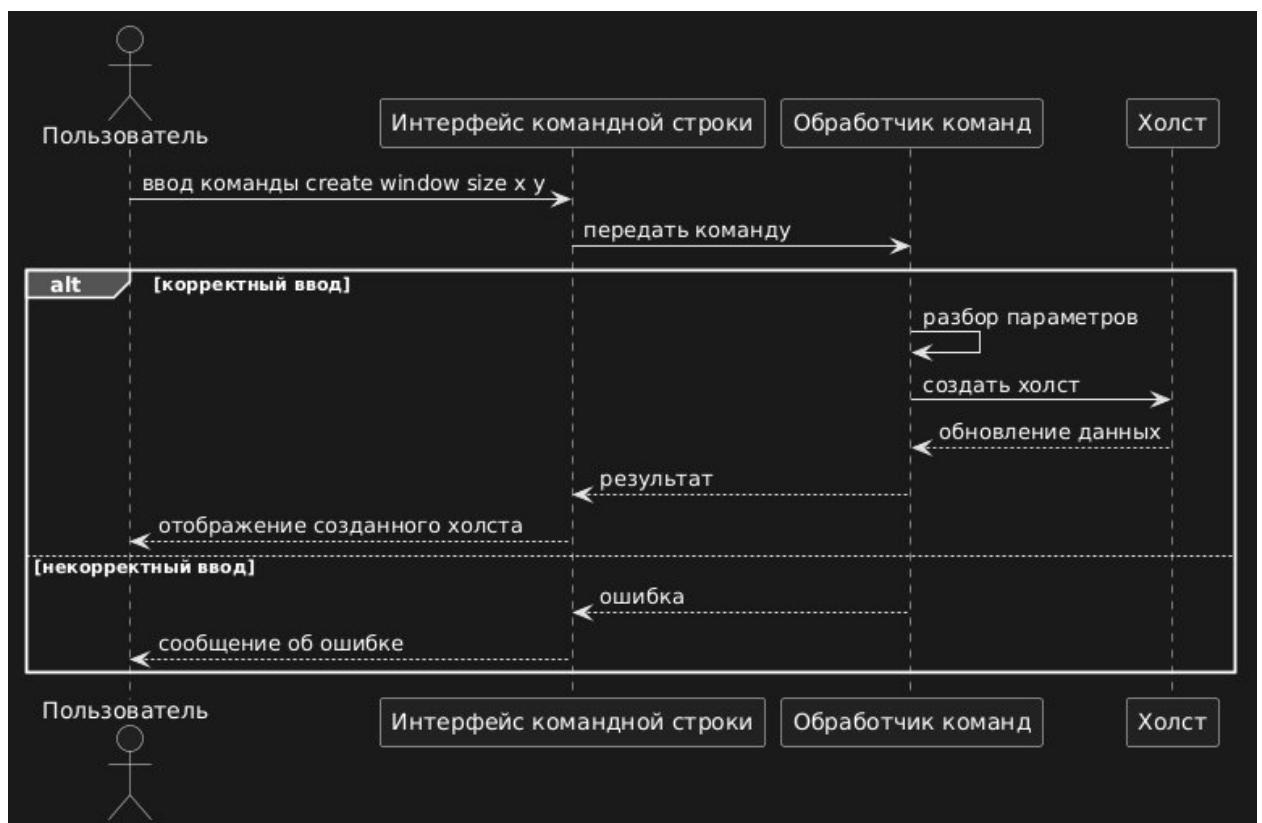
4. Диаграммы UML

4.1 Диаграмма вариантов использования (Use Case Diagram)

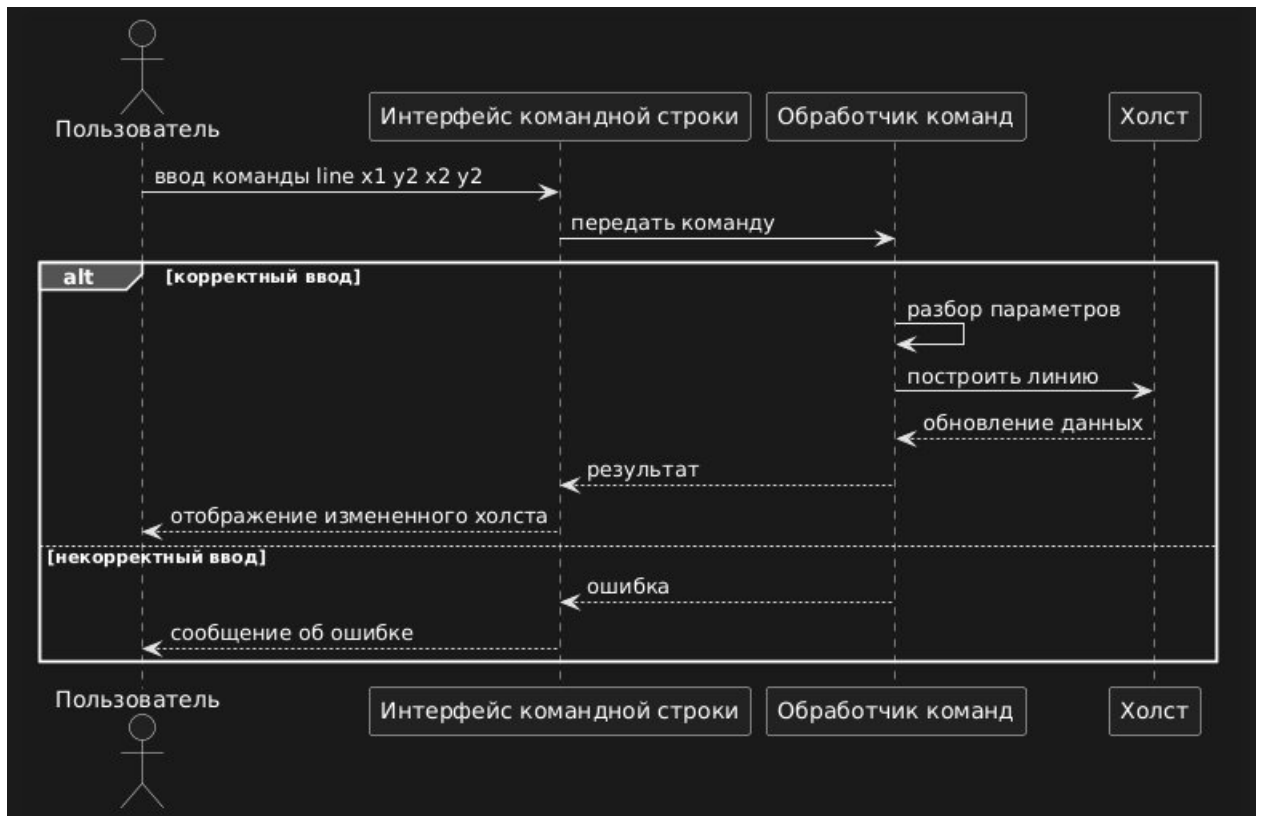


4.2 Sequence diagram

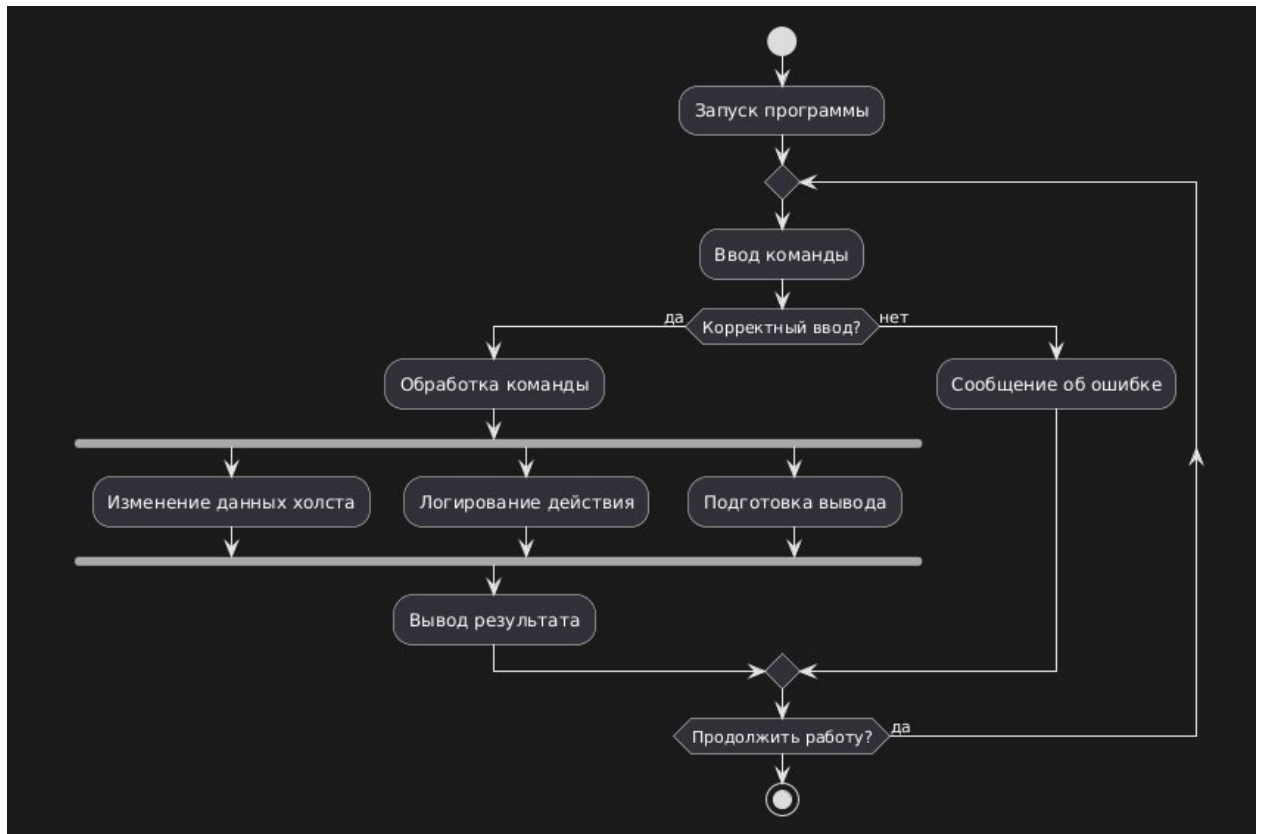
Сценарий «Создание холста»



Сценарий «Создание линии»



4.3 Activity diagram



4.4 Class diagram



5. Паттерны

Для реализации проекта консольного Paint (САПР для ASCII-арта) были выбраны следующие паттерны:

5.1 Паттерн Command

Поведенческий паттерн Command используется для инкапсулирования пользовательских действий в отдельные объекты.

В данном проекте класс `Command()` задает общий интерфейс для всех команд с помощью метода `execute()`. Команды: `DrawLineCommand`, `DrawRectangleCommand`, `FillCommand`, `SaveCommand` и `LoadCommand` реализуют данный метод и содержат параметры, которые необходимы для выполнения операции.

Класс `CommandHandler()` создает объекты команд на основе пользовательского ввода и вызывая их выполнение, передавая объект `Canvas`.

5.2 Паттерн Factory

Порождающий паттерн `Factory` применяется для создания объектов команд.

Роль этого паттерна в данном проекте выполняет класс `CommandHandler()`, который анализирует введенную пользователем строку и на ее основе создает соответствующий объект команды.

Создание объекта инкапсулировано метода `parseCommand`, что позволяет скрыть детали создание объектов и централизовать этот процесс.

5.3 Паттерн Facade

Структурный паттерн `Facade` используется для реализации простого интерфейса к сложной системе.

В данном проекте класса `CommandHadler()` выступает в роли фасада, обеспечивая взаимоотношение пользователя с системой. Он принимает пользовательский ввод, обрабатывает его, создает объекты команд и инициирует их выполнение.

При этом внутренняя логика работы с холстом (`Canvas`) и файлами (`FileManager`) скрыта от пользователя.

6. Тестирование системы

Для проверки корректности работы программы была разработана система `unit-тестов` и тестовых сценариев с использованием библиотеки `GoogleTest`.

Все тесты находятся в папке `/test` в корне проекта и включают: `unit-тесты` для класса `Canvas` и `FileManager` и команд; тестовые сценарии для рисования, сохранения и загрузки, изменения размера холста.

Запустить тесты можно при переходе в папку `build\tests\Debug` и вводе команды: `.\Название_теста.exe`.

6.1 Unit-тесты

6.1.1 Класс Canvas

Для проверки корректности работы основного класса проекта были созданы тесты, которые проверяют создание холста и корректность его размеров.

После запуска тестирования все команды работали корректно и ошибок при выполнении замечено не было.

Тест для класса располагается в папке /unit, в файле test_canvas.cpp

6.1.2 Команды

Для проверки правильности работы всех команд, связанных с заполнением холста, был создан файл, в котором проверяется: рисование точки, рисование линии, рисование прямоугольника, заливка области, очистка холста, создание нового холста и изменение его размеров.

После запуска тестирования все команды сработали корректно и серьезных ошибок не было выявлено.

Тесты команд располагаются в папке /unit, в файле test_commands.cpp

6.1.3 Класс FileManager

Для проверки точности работы класса, отвечающего за работу с файлами, были созданы тесты, проверяющие сохранение холста в файл, загрузка холста из файла, сохранение пустого холста и корректность размеров после загрузки.

После запуска тестирования все команды были выполнены корректно и ошибок в их работе не было обнаружено.

Тесты для класса FileManager располагаются в папке /unit, в файле test_filemanager.cpp

6.2 Тестовые сценарии

6.2.1 Тестирование рисования

Для проверки работы программы в условиях, приближенных к реальному использованию, был написан сценарий, который проверяет последовательное выполнение команд рисования, корректность отображения фигур на холсте и совместную работу нескольких команд.

После запуска тестового сценария явных ошибок в работе программы выявлено не было. Тестирование завершилось успешно.

Тест располагается в папке /scenarios, в файле scenario_draw.cpp

6.2.2 Тестирование сохранения и загрузки

Для проверки работы программы в условиях, близких к реальному использованию, был создан сценарий, проверяющий сохранение данных холста

в файл, загрузку данных из сохраненного файла и сохранение целостных данных.

После выполнения тестового сценария критических ошибок в работе программы выявлено не было. Тестирование завершилось успешно.

Тест располагается в папке /scenarios, в файле scenario_save_load.cpp

6.2.3 Тестирование изменения размера

Для проверки работоспособности программы в условиях, приближенных к реальным, был создан сценарий, который проверяет изменение размеров холста, очистку данных после изменения размера, корректное создание новой сетки.

После выполнения тестового сценария явных ошибок в работе программы не было выявлено. Тестирование завершилось успешно.

Тест располагается в папке /scenarios, в файле scenario_resize.cpp

7. Dockerfile для автоматической сборки проекта

В рамках реализации проекта - консольного приложения для создания ASCII-рисунков были созданы Dockerfile, .dockerignore и docker-compose.yml для автоматической сборки проекта.

В .dockerignore записаны файлы и папки: /build, .git, .vs и .idea, чтобы Docker не копировал лишнее и не нагружал проект.

В Dockerfile прописаны инструкции по для автоматической сборки Docker-образа, чтобы обеспечить полную изоляцию и переносимость приложения.

В конфигурационном файле docker-compose.yml декларативно описано приложение. Он запускает всю инфраструктуру, фиксирует все зависимости и настройки и изолирует проект.

В результате создания этих файлов и запуска Docker проект успешно запустился в окружении Linux, и все тесты проекта прошли успешно.

Ссылка на репозиторий с готовой частью проекта - [itisjustmeeee/Project_for_PKS: new repository for special project for PKS](https://github.com/itisjustmeeee/Project_for_PKS)